

Proyecto libre Diseño de Software

Objetivo

El objetivo de este proyecto es buscar **un caso de uso o varios** en el que, análisis incluido, se detecte la necesidad de aplicar uno o varios patrones combinados **de entre los que se han trabajado en la asignatura**. Esta forma de entregar el proyecto final es optativa entre el proyecto ‘Diseño y desarrollo de un juego de combate’ y el proyecto de casos de uso libre (uno u otro).

Normativa de Realización, Entrega y Evaluación

- Este proyecto también se realizará de forma grupal (se espera que el número de alumnos esté comprendido entre 3 ó 4 alumnos por grupo). No se permiten proyectos de forma individual. No se espera una interfaz visual sino una adecuada programación por consola que demuestre la aplicación correcta de los patrones detectados. El proyecto se realizará únicamente con el lenguaje de programación **Java**.
- **La entrega deberá hacerse mediante *Blackboard* antes del 11/05/2026** a las 15:00 horas (hora peninsular en España), y la presentación se hará a partir de ese mismo día en clase de forma presencial.
- La entrega se compondrá de un único fichero comprimido que contendrá el proyecto exportado de eclipse, o el código fuente si se hace con otro IDE, y la memoria. **El nombre del fichero será el nombre de los alumnos y grupo según la formación de los grupos haya sido consensuada en clase.**
- Cualquier sospecha de copia entre dos o más proyectos derivará en la calificación de 0 para todos los alumnos involucrados.
- Los proyectos que den errores de compilación no obtendrán ningún punto.
- **No se admite como entrega un caso de uso exclusivamente del patrón *Singleton***, pero sí puede utilizarse en combinación con otros.
- La nota de este proyecto **cuenta el 25% de la nota final de la evaluación**, tanto en la convocatoria ordinaria como en la extraordinaria, distribuida como sigue:
 - Descripción general del caso de uso: (0,5 puntos)
Contexto, restricciones, alcance
 - Identificación de los componentes del caso de uso
Componentes abstractos y concretos (2 puntos)
Diagrama UML de cada patrón (2 puntos)
 - Código fuente (0,5 punto)
Incluye la **clase Test y los resultados obtenidos por consola**.
 - Conclusiones y próximos pasos sobre el caso de uso (0,5 punto)
 - Uso de camelCase y legibilidad (0,5 punto)

- Si el proyecto consta de la aplicación de 1 ó 2 patrones, entonces la máxima puntuación será de 6 puntos.
- Número de patrones aplicados correctamente **con los puntos anteriores**
 - se consiguen 3 patrones (+1 punto)
 - se consiguen 4 patrones (+1 punto)
 - se consiguen 5 patrones (+1 punto)
 - se consiguen 6 o más patrones (+1 punto)

Proyecto *DesignPatterns*

Diseño y Desarrollo de un Juego de Combate

Introducción

El objetivo de este proyecto es desarrollar un juego de ordenador en el que el protagonista vaya combatiendo contra los enemigos que se va encontrando, utilizando para ello diferentes armas, tipos de ataque, etc.

Dinámica del Juego

- El juego se juega por consola, por lo que no tendrá interfaz gráfica. Pero si alguien sabe, y/o quiere, puede hacerla.
- Cada combate se resolverá por turnos: se elige al azar quién ataca primero, y a partir de ese momento el jugador y la máquina se van turnando.
- En cada turno, el jugador podrá elegir entre una serie de acciones disponibles (diferentes tipos de ataques, armas, defenderse, no hacer nada...).
- Tras cada ataque, el programa evaluará los daños causados al oponente. Pierde el combate aquel contrincante que primero se quede sin vida.
- La evaluación de los daños se realizará de manera algorítmica (podéis inventar la fórmula que queráis para el reparto de los puntos).
- El juego continúa combate tras combate, contra distintos tipos de enemigos, hasta que el jugador pierde. Con cada combate ganado el jugador acumula puntos.
- Los combates se desarrollarán en distintos mundos/escenarios.

Personajes

- Todos los personajes, tanto el jugador como los enemigos, tendrán una serie de atributos que afectarán a los resultados de los combates (fuerza, resistencia, agilidad...).
- Al principio de la partida el jugador definirá los valores que su personaje va a tener para cada atributo.
- Habrá distintas categorías de enemigos (guerreros, hechiceros, mutantes...), cada uno con sus características propias.
- Las mismas categorías aplican en todos los mundos/escenarios, pero en cada uno de dichos mundos cada categoría de enemigo presenta características diferentes (por ejemplo, los brujos de la selva podrían hacer vuelos cortos y los brujos del mar vuelos largos).
- Cada enemigo se definirá también por una determinada estrategia, de entre varias disponibles (defensiva, agresiva...).
- Un personaje puede pasar por varios estados a lo largo de un combate (herido, paralizado, activo...). Pasará de un estado a otro como consecuencia de los distintos eventos que vayan ocurriendo a lo largo del combate (pérdida de mucha vida, recuperación de vida por consumo de alguna poción, paralizado por culpa de un hechizo,...).

•

Tareas a realizar

Al final del cuatrimestre se deberá entregar un juego funcionalmente completo que incluya como mínimo los siguientes desarrollos:

1. **(1 punto)** Se deberá utilizar el patrón **Strategy** para gestionar el conjunto de estrategias posibles y la estrategia que en cada combate adopte el enemigo de turno.
2. **(1 punto)** Se deberá emplear un patrón **Decorator** para gestionar las distintas acciones a realizar por los personajes: se decorará la acción básica con los modificadores adecuados dependiendo de la estrategia y las características del personaje. Por ejemplo, si el personaje realiza un 'ataque con espada', este ataque habrá que decorarlo para hacerlo un 'ataque poderoso con espada' si el personaje tiene un alto valor de su atributo Fuerza.
3. **(1 punto)** Se deberá emplear un patrón **State** para controlar el estado de los personajes durante el combate, de forma que los personajes vayan pasando de un estado a otro dependiendo de las incidencias del combate.
4. **(2 puntos)** Se deberá emplear un patrón **Abstract Factory** para crear los diferentes enemigos. Existirán varios tipos de enemigos, comunes a todos los mundos/escenarios pero adaptados a cada mundo/escenario.
5. **(1 punto)** Se deberá emplear un patrón **Singleton** en la clase encargada de hacer los cálculos sobre el resultado de los ataques, para que sólo haya una instancia de dicho 'calculador' en el sistema.
6. **(1 punto)** Se deberá emplear un patrón **Template Method** para implementar en cada clase *Enemigo* el algoritmo para decidir cuál será la siguiente acción a realizar.
7. **(1 punto)** Se deberá usar un patrón **Facade** para la clase *GameController*.
8. **(1 punto)** Se deberá realizar un documento de memoria que recoja:
 - La descripción del juego.
 - Un pequeño manual de uso.
 - El diagrama de clases de cada patrón empleado, convenientemente explicado.
9. **(1 punto)** Se reserva esta puntuación para la claridad, sencillez y estilo del código. Se valorarán los comentarios, indentación, etc.

Normativa de Realización, Entrega y Evaluación

- El proyecto se realizará por equipos. Cada equipo estará formado por 3-4 alumnos como máximo. Puede hacerse si se desea con Interfaz Gráfica de Usuario, pero no supondrá necesariamente un incremento en la nota.
- El proyecto se realizará únicamente con el lenguaje de programación Java.
- La entrega deberá hacerse mediante *Blackboard* **antes del 11/05/2026 a las 15:00 horas** (hora peninsular en España), y la presentación se hará ese mismo día en clase de forma presencial.
- La entrega se compondrá de un único fichero comprimido que contendrá el proyecto exportado de eclipse, o el código fuente si se hace con otro IDE, y la memoria. **El nombre del fichero será el nombre del equipo y deberán subirlo todos sus integrantes.**
- El último apartado puntuará si, y sólo si, al menos algún otro apartado se ha resuelto correctamente.
- Los apartados puntúan como un todo. Si un apartado se resuelve al completo correctamente, el alumno obtendrá los puntos asignados a dicho apartado. En cualquier otro caso, no obtendrá ningún punto.
- Cualquier sospecha de copia entre dos o más prácticas derivará en la calificación de 0 para todos los alumnos involucrados.
- Los proyectos que den errores de compilación no obtendrán ningún punto.
- La nota del proyecto **cuenta el 25% de la nota final de la evaluación**, tanto en la convocatoria ordinaria como en la extraordinaria.